Laboratório Prolog · Aula 13

Preparação: Programar descrevendo, não instruindo

Previsão: O que paradigma(L, funcional) e paradigma(java, logico) vão responder?

paradigma(scheme,funcional) e false

Saída do ola.pl

Output:

```
?- linguagem(A).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)
?- paradigma(prolog,A).
   paradigma(prolog,logico)
?- paradigma(A,funcional).
   paradigma(scheme,funcional)
?- paradigma(java,logico).
   false.
?- linguagem(A).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)
?- paradigma(prolog,A).
   paradigma(prolog,logico)
?- paradigma(A,funcional).
   paradigma(scheme,funcional)
?- paradigma(java,logico).
   false.
?- linguagem(A).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)
```

0.1 Qual é a diferença entre paradigma(prolog, P) e paradigma(prolog, logico)? O que cada uma pergunta?

paradigma(prolog, P): instancia a variável P com o valor encontrado e é uma pergunta aberta por instanciação/consulta com retorno de valor.
paradigma(prolog, logico): a afirmação com a base de dados e responde com um booleano (true se estiver cadastrado, ou false se não estiver). É uma pergunta fechada de verificação de existência (verdadeiro ou falso).

0.2 paradigma(java, logico) deu false: Java não é lógico ou o programa não sabe isso? Justifique.

O retorno false não significa que a linguagem Java é fisicamente ou conceitualmente incapaz de usar lógica (ou que ela não tenha relação com lógica), mas sim que a base de conhecimento do Prolog não contém esse fato cadastrado

Etapa 1: Uma base de fatos e perguntas sobre ela

Previsão: Quantas respostas terá mulher(Quem)? E quais pares terá (mae(M, F), mulher(F))?

mulher(Quem): 7 mae(M, F), mulher(F): 4 pares.
mae(lucia,tereza),mulher(tereza) mae(marta,joana),mulher(joana)
mae(tereza,bia),mulher(bia) mae(joana,ana),mulher(ana)

Saída do familia.pl

```
Output:
--- 1. sem variavel: a resposta e true ou false
?- mae(joana,jake).
   mae(joana,jake)
?- mae(jake,joana).
   false.
?- pai(maria,jake).
   false.
--- 2. com variavel: o Prolog procura os valores que tornam a consulta verdadeira
?- mae(joana,A).
   mae(joana,jake)
   mae(joana,ana)
?- pai(A,jake).
   pai(jorge,jake)
?- mulher(A).
   mulher(lucia)
   mulher(marta)
   mulher(tereza)
   mulher(joana)
   mulher(bia)
   mulher(ana)
   mulher(dora)
--- 3. conjuncao (virgula = E): as duas metas valem com o mesmo valor
?- mae(joana,A),homem(A).
   mae(joana,jake),homem(jake)
?- pai(vern,A),mulher(A).
   pai(vern,joana),mulher(joana)
?- mae(A,B),mulher(B).
   mae(lucia,tereza),mulher(tereza)
   mae(marta,joana),mulher(joana)
   mae(tereza,bia),mulher(bia)
   mae(joana,ana),mulher(ana)
```

1.1 Escreva em português o que significa a consulta (mae(Mae, Filha), mulher(Filha)) e quantas respostas ela teve.

Em português, essa consulta significa: "Quem é a mãe (Mae) e a filha (Filha), de modo que Filha seja do sexo feminino?" (ou, de forma simplificada: "Quais são os pares de mães e suas respectivas filhas?"). Ela teve 4 respostas.

1.2 pai(maria, jake) deu false mas a base não menciona maria. Que pressuposto explica? Por que pode ser um problema?

O Prolog assume que a sua base de dados contém todo o conhecimento existente e verdadeiro sobre aquele universo. Se uma informação não está escrita ali (como o fato de maria existir ou ser mãe), o Prolog não assume que ela é "desconhecida"; ele assume categoricamente que ela é falsa. No mundo real, a ausência de uma informação em um sistema de computação nem sempre significa que ela não existe.

1.3 Em que ordem as respostas de mulher(Quem) apareceram? O que determina essa ordem?

As respostas apareceram exatamente na seguinte ordem: lucia, marta, tereza, joana, bia, ana e dora. O que determina essa ordem é a ordem física textual de declaração na base de fatos (de cima para baixo). O Prolog realiza a sua busca (matching) sequencialmente, varrendo o arquivo do código-fonte a partir da primeira linha e avançando para as próximas conforme encontra termos que satisfazem a consulta.

Etapa 2: Regras: ensinando o Prolog a deduzir

Previsão: Quais respostas terão tio(X, jake) e primo(leo, X)?

tio(X, jake) terá 1 resposta: tio(paulo, jake). Explicação: Os pais de jake são joana e jorge. O irmão de joana que é homem é paulo. primo(leo, X) terá 2 respostas: primo(leo, jake) e primo(leo, ana). Explicação: O pai de leo é paulo. O irmão de paulo é joana. Os filhos de joana são jake e ana.

Previsão: Quais unificações vão falhar: f(X,X)=f(a,b), f(X,X)=f(a,Y), [A|B]=[], 2+3=5?

Quais unificações vão falhar: • f(X, X) = f(a, b) (Falha) • [A|B] = [] (Falha) • 2 + 3 = 5 (Falha)

Saídas de regras.pl e unificacao.pl

```
--- avo: duas regras progenitor encadeadas
?- avo(A,jake).
   avo(marta,jake)
   avo(vern,jake)
?- avo(osvaldo,A).
   avo(osvaldo,joana)
   avo(osvaldo,paulo)
   avo(osvaldo,bia)
--- irmao, tio e primo
?- irmao(jake,A).
   irmao(jake,ana)
?- tio(A,jake).
   tio(paulo,jake)
?- primo(leo,A).
   primo(leo,jake)
   primo(leo,ana)
--- ancestral (recursiva)
?- ancestral(osvaldo,A).
   ancestral(osvaldo,vern)
   ancestral(osvaldo,tereza)
   ancestral(osvaldo,joana)
   ancestral(osvaldo,paulo)
   ancestral(osvaldo,jake)
   ancestral(osvaldo,ana)
   ancestral(osvaldo,leo)
   ancestral(osvaldo,bia)
?- ancestral(A,jake).
   ancestral(joana,jake)
   ancestral(jorge,jake)
   ancestral(lucia,jake)
   ancestral(marta,jake)
   ancestral(osvaldo,jake)
   ancestral(vern,jake)
?- ancestral(jake,A).
   false.
```

2.1 Por que ancestral precisa de duas cláusulas? Qual é o caso base e qual é o caso recursivo?

• Caso Base (ancestral(X, Y) :- progenitor(X, Y).): Define que um progenitor direto (pai ou mãe) é um ancestral. Ele interrompe a busca recursiva. • Caso Recursivo (ancestral(X, Y) :- progenitor(X, Z), ancestral(Z, Y).): Define que se X é progenitor de alguém (Z), e esse Z é ancestral de Y, então X também é ancestral de Y. Ele serve para "subir" ou "descer" degraus na árvore geracional (avós, bisavós, etc.).

2.2 Apague X \== Y da regra irmao e consulte irmao(jake, X). O que mudou? Por que a meta é necessária?

• O que mudou: A consulta passou a retornar que jake é irmão de si mesmo (irmao(jake, jake)). • Por que é necessária: A meta X \== Y garante a desigualdade, ou seja, impede que o Prolog unifique as duas variáveis com a mesma pessoa. Sem ela, o Prolog entende que se M é mãe de X e M é mãe de Y, X e Y podem ser o mesmo indivíduo.

2.3 Por que f(X,X)=f(a,b) não unifica mas f(X,X)=f(a,Y) unifica? Quais valores ficam em X e Y?

• f(X, X) = f(a, b) falha porque a variável X não pode valer a e b ao mesmo tempo (o Prolog exige consistência na mesma variável). • f(X, X) = f(a, Y) funciona porque o Prolog primeiro casa o primeiro argumento atribuindo X = a. Ao olhar o segundo argumento, como o primeiro X já vale a, ele descobre que a = Y.

2.4 avo responde em várias direções. Isso é possível numa função de Java ou Scheme? Explique a diferença.

Em linguagens como Java ou Scheme, as funções são direcionais (unidirecionais). Você precisa definir rigidamente o que é dado de entrada (input) e o que é retorno (output). Para fazer as duas coisas, você seria obrigado a escrever duas funções distintas (ex: getAvos(pessoa) e getNetos(pessoa)).

Etapa 3: Listas: o mesmo padrão do Scheme, em formato de relação

Previsão: O que concatena(X, Y, [1,2,3]) vai responder? Quantas respostas?

A consulta concatena(X, Y, [1, 2, 3]) vai responder todas as formas possíveis de dividir a lista [1, 2, 3] em duas partes (X e Y), gerando um total de 4 respostas. As soluções exatas que aparecem na tela são: 1. X = [], Y = [1, 2, 3] 2. X =, Y = [2, 3] 3. X =, Y = [3] 4. X =, Y = []

Saída do listas.pl

```
--- membro: serve para testar E para gerar
?- membro(b,[a,b,c]).
   membro(b,[a,b,c])
?- membro(z,[a,b,c]).
   false.
?- membro(A,[a,b,c]).
   membro(a,[a,b,c])
   membro(b,[a,b,c])
   membro(c,[a,b,c])
--- tamanho
?- tamanho([a,b,c],A).
   tamanho([a,b,c],3)
?- tamanho([],A).
   tamanho([],0)
```

```
--- concatena: o mesmo codigo, em varias direcoes
?- concatena([1,2],[3,4],A).
   concatena([1,2],[3,4],[1,2,3,4])
?- concatena(A,[3],[1,2,3]).
   concatena([1,2],[3],[1,2,3])
?- concatena([1],A,[1,2,3]).
   concatena([1],[2,3],[1,2,3])
?- concatena(A,B,[1,2,3]).
   concatena([],[1,2,3],[1,2,3])
   concatena([1],[2,3],[1,2,3])
   concatena([1,2],[3],[1,2,3])
   concatena([1,2,3],[],[1,2,3])
--- inverte
?- inverte([1,2,3],A).
   inverte([1,2,3],[3,2,1])
```

3.1 Em tamanho, qual é o caso base e qual é o passo recursivo? O que fica pendente a cada chamada?

• Caso Base (tamanho([], 0).): Diz que uma lista vazia tem tamanho zero. Ele interrompe a recursão. • Passo Recursivo (tamanho([_|Cauda], N) :- tamanho(Cauda, M), N is M + 1.): Ignora a cabeça, calcula recursivamente o tamanho da cauda (M) e incrementa o total (N is M + 1). • O que fica pendente: A cada chamada recursiva, a computação da soma aritmética N is M + 1 fica empilhada (pendente) na memória. O Prolog só consegue resolver essas adições de trás para frente quando finalmente atinge o caso base e começa a desempilhar os valores de M.

3.2 Por que concatena(X,Y,[1,2,3]) tem 4 respostas? Compare com anexar do Scheme: o que ele não consegue fazer?

• Por que tem 4 respostas: Como a lista possui 3 elementos, existem exatamente 4 "pontos de corte" possíveis para dividi-la (antes do 1, entre o 1 e o 2, entre o 2 e o 3, e após o 3). O Prolog testa todas essas

possibilidades por meio do backtracking. • Comparação com o Scheme: Uma função como append em Scheme (ou na maioria das linguagens funcionais/imperativas) é unidirecional. Ela exige que as listas de entrada X e Y sejam fornecidas para construir e retornar uma nova lista. O Scheme não consegue trabalhar ao contrário, ou seja, ele é incapaz de receber uma lista pronta e deduzir quais entradas poderiam tê-la gerado.

3.3 membro(X,[a,b,c]) gerou a, b e c sem laço. Que mecanismo do Prolog fez isso?

O mecanismo responsável por isso é o Backtracking combinado com a Unificação. 1. Na primeira tentativa, o Prolog casa a consulta com o caso base membro(X, [X|_]), unificando X com a cabeça da lista (a). 2. Quando o sistema busca por mais respostas (ou o utilitário força isso), o Prolog faz o backtracking (retorna ao ponto de escolha anterior) e tenta a segunda cláusula: membro(X, [_|Cauda]). 3. Isso faz com que a cabeça antiga seja descartada e o Prolog chame membro(X, [b, c]). O processo se repete de forma puramente lógica e declarativa, sem a necessidade de estruturas de repetição como for ou while.

Etapa 4: Aritmética: onde o Prolog deixa de ser relacional

Previsão: O que mostram X = 2 + 3 e X is 2 + 3? E fatorial(N, 120)?

• X = 2 + 3 mostra X = 2 + 3 (unificação puramente estrutural). • X is 2 + 3 mostra X = 5 (o lado direito é calculado antes da unificação). • fatorial(N, 120) gera um ERRO: instantiation_error (o Prolog falha ao tentar avaliar o teste de maioridade N > 0 com uma variável desconhecida).

Saída do aritmetica.pl

```
--- = unifica, is calcula
?- A=2+3.
   2+3=2+3
?- A is 2+3.
   5 is 2+3
?- 5=:=2+3.
   5=:=2+3
?- 2+3=5.
   false.
?- A is 7 mod 3.
   1 is 7 mod 3
?- A is 7//2.
   3 is 7//2
?- A is 2^10.
   1024 is 2^10
```

```
--- fatorial, fib e soma
?- fatorial(5,A).
   fatorial(5,120)
?- fatorial(0,A).
   fatorial(0,1)
?- fib(10,A).
   fib(10,55)
?- soma([1,2,3,4],A).
   soma([1,2,3,4],10)
--- aritmetica nao e relacional
?- fatorial(A,120).
   ERRO: instantiation_error
?- A is B+1.
   ERRO: instantiation_error
```

4.1 Explique com exemplos a diferença entre =, is e =:=.

• = (Unificação Estrutural): Não faz cálculos matemáticos. Apenas verifica se os dois lados possuem a mesma estrutura ou se variáveis podem ser ligadas para torná-los idênticos. • Exemplo: X = 2 + 3 resulta em X = 2 + 3. O Prolog não soma. • Exemplo: 5 = 2 + 3 retorna false. • is (Avaliação Aritmética): Avalia matematicamente a expressão do lado direito e tenta unificar o resultado numérico final com o termo do lado esquerdo. • Exemplo: X is 2 + 3 resulta em X = 5. • =:= (Igualdade Aritmética): Avalia matematicamente ambos os lados (esquerdo e direito) e compara se os resultados numéricos finais são iguais. Não faz atribuição/unificação de variáveis. • Exemplo: 2 + 3 =:= 4 + 1 retorna true. • Exemplo: 5 =:= 2 + 3 retorna true.

4.2 Por que fatorial(N,120) dá erro e concatena(X,[3],[1,2,3]) funcionou? O que há de diferente na aritmética?

A grande diferença é que a aritmética clássica no Prolog não é relacional. • Na Etapa 3, concatena opera usando apenas unificação de estruturas de listas ([] e [H|T]). O Prolog consegue preencher variáveis vazias tanto na entrada quanto na saída de forma bidirecional. • No caso do fatorial(N, 120), a execução esbarra imediatamente na meta N > 0 (ou em N1 is N - 1). Operações aritméticas e operadores de comparação (como >, <, is) exigem obrigatoriamente que todas as variáveis do lado direito estejam instanciadas (com valores numéricos concretos conhecidos). Como N começa vazio, o interpretador dispara um erro de instanciação por não conseguir processar a lógica matemática de trás para frente.

4.3 Compare o fatorial em Prolog com o fact em Haskell: o que é igual e o que é diferente?

• O que é igual: • Ambos usam casamento de padrões (pattern matching) para definir o caso base (ex: fatorial(0, 1) no Prolog e fact 0 = 1 no Haskell). • Ambos utilizam recursão pura para solucionar problemas repetitivos ao invés de laços como for ou while. • O que é diferente: • Função vs. Relação: Em Haskell, fact é uma função matemática real que recebe uma entrada e

produz um retorno direto (fact 5). Em Prolog, fatorial é um predicado/relação de aridade 2 onde o resultado é unificado com o segundo argumento (fatorial(5, F)). • Direcionalidade: Embora o Prolog sofra restrições aritméticas no is, ele foi conceitualmente desenhado para não ter distinção nativa entre variáveis de entrada e saída. Já o Haskell possui assinaturas de tipos rígidas e estritas de entrada/saída (Int -> Int). • Avaliação: O Haskell usa avaliação preguiçosa (lazy evaluation), adiando cálculos matemáticos até que sejam estritamente necessários. O Prolog avalia expressões aritméticas de forma imediata assim que encontra o predicado is.

Etapa 5: Controlando (e sofrendo com) a busca

Previsão: qual de solteiro1/solteiro2 dá false? maximo2(5,3,M) uma ou duas respostas? maximo3(5,3,3) true ou false?

• solteiro1(X) vs solteiro2(X): A consulta solteiro2(_) dará false. (enquanto solteiro1 encontra os homens solteiros). • maximo2(5, 3, M): Retornará duas respostas (M = 5 e M = 3). • maximo3(5, 3, 3): Dá true. (o que é semanticamente incorreto para um algoritmo de máximo).

Saída de controle.pl e mensagem de ancestral_ruim

```
--- A) backtracking: acompanhe a ordem das tentativas
?- mesmo_gosto(X, Y).
   tenta ana e ana
   tenta ana e bia
   tenta ana e caio
   resposta: mesmo_gosto(ana,caio)
   tenta bia e ana
   tenta bia e bia
   tenta bia e caio
   tenta caio e ana
   resposta: mesmo_gosto(caio,ana)
   tenta caio e bia
   tenta caio e caio
```

```
Output:
ancestral_bom(osvaldo, jake)?
  sim
ancestral_ruim(osvaldo, jake)? (vai estourar a pilha)

Fatal Error: local stack overflow (size: 16384 Kb, reached: 16383 Kb, environment variabl
```

Previsão: Quantas colorações existem com 3 cores? E com 2 cores?

• Existem 0 colorações com 2 cores (pois PR, SP e MS formam um triângulo de fronteiras mútuas e exigem 3 cores distintas). Com 3 cores, o Prolog encontrará 24 colorações.

Saída do mapa.pl

Output:

```
--- coloracoes com 3 cores, na ordem [RS, SC, PR, SP, MS, MG, RJ, ES]
[vermelho,verde,vermelho,verde,azul,vermelho,azul,verde]
[vermelho,verde,vermelho,azul,verde,vermelho,verde,azul]
--- quantas coloracoes existem com 3 cores?
24
--- e com 2 cores?
?- mapa([vermelho,verde],A).
   false.
--- e com 4 cores?
1728
```

5.1 Com o rastro do bloco A, explique como o Prolog encontrou as respostas de mesmo_gosto(X,Y). Que meta falhou e para onde voltou?

O Prolog tenta combinar os amigos gerados por amigo(X) e amigo(Y). 1. Ele testa primeiro X = ana e Y = ana. A meta de desigualdade X \== Y falha. 2. Ao falhar, o Prolog faz o backtracking para a escolha pendente mais recente: amigo(Y). Ele avança para Y = bia. 3. Ele testa X = ana e Y = bia. Passa por X \== Y. Porém, ao testar os gostos (gosta(ana, cafe) e gosta(bia, cha)), não há unificação para o mesmo tipo de bebida (cafe \== cha). Falha. 4. O Prolog volta a amigo(Y) e tenta Y = caio. Avalia X = ana e Y = caio. A meta de desigualdade passa. gosta(ana, cafe) e gosta(caio, cafe) unificam com B = cafe. Encontra a 1ª resposta: ana e caio. 5. O processo continua reiniciando Y. Quando X = caio e Y = ana, as metas passam novamente. Encontra a 2ª resposta: caio e ana.

5.2 Por que solteiro2 dá false e solteiro1 não? Por que solteiro1 afirma que osvaldo é solteiro?

• Por que solteiro2(X) dá false? A meta \+ casado(X) é executada com X ainda sem valor (livre). O Prolog tenta provar casado(X). Como existem casados na base (vern, jorge), casado(X) é considerado verdadeiro para o universo. A negação de algo verdadeiro é falsa. Portanto, a consulta morre ali mesmo, antes de chegar em homem(X). No solteiro1(X), primeiro homem(X) gera um valor concreto para X (ex: osvaldo), permitindo que \+ casado(osvaldo) filtre corretamente. • Por que afirma que osvaldo é solteiro? Porque o predicado avalia estritamente o fato casado(X). Como o fato casado(osvaldo) não está explicitamente escrito na base de dados, pela Hipótese do Mundo Fechado, o Prolog assume que é falso que ele seja casado. O sistema ignora o conceito biológico/social de que pais geralmente são ou foram casados.

5.3 maximo3(5,3,3) deu true. Explique por que o corte não protegeu esse uso. Como você corrigiria?

• Por que falhou em proteger? Quando você chama maximo3(5, 3, 3), o Prolog tenta casar com a primeira cláusula: maximo3(X, Y, X). Ele tenta unificar as posições: X = 5, Y = 3 e o terceiro argumento X = 3. Como 5 = 3 é uma mentira lógica, a unificação da cabeça da primeira regra falha imediatamente. • Como a cabeça falhou, o interpretador sequer entra no corpo da regra e nunca atinge o corte (!). Ele simplesmente pula para a segunda cláusula maximo3(_, Y, Y)., que unifica perfeitamente com maximo3(5, 3, 3) (pois Y = 3), retornando true. • Como corrigir (Uso do corte verde/verde-amarelo): Deve-se desatrelar a unificação da resposta da cabeça da regra, fazendo-a apenas após o corte garantir que aquela é a única rota válida:

5.4 Por que ancestral_ruim estoura a pilha e ancestral_bom não, se dizem a mesma coisa logicamente?

• O Prolog processa as metas estritamente da esquerda para a direita. • Em ancestral_ruim(X, Y) :- ancestral_ruim(X, Z), ..., a primeiríssima coisa que o interpretador faz ao tentar resolver o problema é chamar a si mesmo com os mesmos parâmetros desconhecidos. Ele entra em uma espiral infinita de auto-chamadas (ancestral_ruim chama ancestral_ruim que chama ancestral_ruim...), sem nunca consumir um fato real da base de dados para reduzir o problema, causando o estouro de pilha (stack overflow). • Em ancestral_bom, a meta progenitor(X, Z) é avaliada primeiro. Ela consome um fato concreto da base de dados. Isso reduz o tamanho do espaço de busca a cada passo recursivo antes de chamar ancestral_bom(Z, Y), garantindo que o programa eventualmente atinja o caso base e termine.

5.5 Quantas colorações com 3, 2 e 4 cores? Por que o programa é declarativo apesar de fazer busca?

• 3 cores: 24 colorações. • 2 cores: 0 colorações (false.). • 4 cores: 432 colorações. • Por que é declarativo? Porque em nenhum momento o programador escreveu algoritmos de ordenação, laços de repetição (while/for), condicionais complexas ou estratégias de atribuição de variáveis para testar os estados do mapa. O código limita-se a descrever as propriedades lógicas e restrições geométricas do problema (as regiões e a obrigatoriedade de desigualdade entre vizinhos). Toda a lógica de busca, controle de pilha e tentativas de caminhos alternativos é terceirizada para o motor de inferência nativo da linguagem.

Fechamento: O que você viu e onde isso se encaixa

6.1 Explique a diferença entre o paradigma lógico e o funcional usando concatena e anexar.

O paradigma funcional (Scheme) computa valores de forma unidirecional, exigindo dados de entrada para produzir um único resultado determinado (anexar só une duas listas conhecidas), enquanto o paradigma lógico

(Prolog) expressa relações multidirecionais por meio de unificação, permitindo que o mesmo código (concatena) descubra tanto o resultado final quanto as partes que o compõem.

6.2 Cite duas deficiências do Prolog que você observou, ligando cada uma a um programa.

• Aritmética não relacional e sensível à ordem das metas: Operações matemáticas não aceitam variáveis livres e quebram a bidirecionalidade lúdica da linguagem, como visto em fatorial(N, 120), que dispara um erro de instanciação ao tentar avaliar N > 0 antes de N possuir um valor numérico fixado. • Vulnerabilidade à recursão infinita (Sensibilidade à ordem das regras): O motor de busca da linguagem é puramente sequencial (de cima para baixo, esquerda para a direita). Isso faz com que definições logicamente corretas, mas mal ordenadas estoure a pilha de memória, como visto no travamento por stack overflow do programa ancestral_ruim.

6.3 Para que tipo de problema você escolheria Prolog em vez de Java? E para qual não escolheria?

• Escolheria Prolog para: Sistemas de Inteligência Artificial Simbólica, motores de regras de negócio complexas, e problemas de satisfação de restrições combinatorialmente exaustivos (como o exemplo de colorir o mapa da Etapa 5B ou solucionar quebra-cabeças lógicos). • Justificativa: Nesses problemas, modelar todas as condicionais e laços de busca manual em Java gera um código verboso e propenso a erros, enquanto no Prolog basta descrever as regras declarativamente e deixar o motor de backtracking fazer o trabalho duro. • Não escolheria Prolog para: Sistemas voltados a processamento intensivo de dados, desenvolvimento web, aplicativos móveis ou softwares com muita concorrência e manipulação de arquivos. • Justificativa: O Prolog peca em desempenho computacional bruto e ferramentas de ecossistema para essas áreas, tarefas onde o Java brilha pelo controle estrito de tipos, gerenciamento de threads, eficiência em memória e bibliotecas robustas de infraestrutura.